

컴퓨터비전_인공지능(AI) 개론

2. Numpy / Matplotlib

NumPy 배열과 연산

- NumPy 배열이 텐서의 기초가 되는 이유를 이해합니다.
- 반복문 대신 벡터화된 연산을 통해 성능을 최적화하는 방법을 배웁니다.

데이터 해석과 시각화

- 축, 집계, 필터링을 모델 연산(Loss 등)과 연결하여 이해합니다.
- Matplotlib을 사용하여 데이터를 시각화하고 분포를 해석하는 능력을 기릅니다.



"오늘은 NumPy 문법 암기가 아니라, 데이터를 배열 단위로 사고하는 법을 익히는 것이 핵심입니다."

넘파이는 왜 필요한가?

딥러닝 데이터 흐름

- 1 Raw Data**
이미지, 텍스트, 센서 데이터 등 다양한 원본 데이터
- 2 숫자 변환 (Tensors)**
모델이 이해할 수 있는 다차원 배열(숫자) 형태로 변환
- 3 행렬 연산**
가중치와 입력 데이터 간의 대규모 행렬 곱셈 수행
- 4 예측 결과**
최적화된 확률값 또는 분류 결과 출력

NumPy의 핵심 장점

-  **인접 메모리 구조**
메모리에 연속적으로 저장되어 CPU 캐시 효율 극대화
-  **벡터화 연산 (Vectorization)**
느린 파이썬 for 루프 없이 전체 배열을 한 번에 계산
-  **C 언어 백엔드**
핵심 연산이 최적화된 C 코드로 실행되어 고속 처리
-  **브로드캐스팅 (Broadcasting)**
크기가 다른 배열 간의 연산을 자동으로 수행

Python List → NumPy → PyTorch

딥러닝을 위한 데이터 컨테이너의 발전 과정을 이해합니다.



🔑 핵심: PyTorch의 모든 기능은 NumPy의 설계 철학을 계승하고 확장했습니다.

☰ Python List

❗ 이질적 데이터 (Heterogeneous)

숫자, 문자열 등 다양한 타입 혼합 가능

✂ 흩어진 메모리 (Scattered)

객체의 주소(Pointer)를 저장하여 참조

🔄 명시적 반복문 (Loop)

for문을 통한 느린 순차 처리

메모리 구조 (참조)

ref

Obj

≡ NumPy Array

✅ 동질적 데이터 (Homogeneous)

단일 데이터 타입(int32, float64 등)만 허용

🏠 인접 메모리 (Contiguous)

연속된 메모리 블록에 값(Value) 직접 저장

⚡ 벡터화 연산 (Vectorized)

CPU의 SIMD 명령어를 활용한 일괄 처리

메모리 구조 (연속)

1

2

3

4

5

Python For Loop (순차 처리)



NumPy Vectorization (병렬 처리)



왜 반복문이 사라지는가?

딥러닝에서는 수백만 개의 데이터를 처리해야 합니다.
Python의 for문은 인터프리터 오버헤드로 인해 느리지만,
NumPy는 C언어 레벨에서 최적화된 병렬 연산을 수행합니다.

SLOW `[x * 2 for x in data]`

FAST `data * 2`



확장성 부족 (Loop)

데이터가 늘어날수록 처리 시간이 선형적으로 급증하여
대규모 딥러닝 학습에 부적합합니다.



스케일 확장성 (Vector)

데이터가 커져도 병렬 처리를 통해
연산 속도를 획기적으로 유지할 수 있습니다.



기본 변환

`np.array([1, 2, 3])`

파이썬 리스트나 튜플을 **ndarray**로 변환합니다.

`dtype='float32'`와 같이 데이터 타입을 명시할 수 있음



특수 초기화

`np.zeros()` / `np.ones()`

0 또는 1로 채워진 배열을 생성합니다. (초기화)

딥러닝 가중치 초기화나 마스크 생성 시 주로 사용됨



범위/수열 생성

`np.arange()` / `linspace()`

특정 구간의 수열이나 등간격 값을 생성합니다.

반복문 없이 벡터화 연산에 사용할 인덱스를 만들 때

ndarray: 데이터 컨테이너

NumPy의 **ndarray**는 단순한 배열이 아니라,
데이터의 형태(**Shape**)와 타입(**Dtype**)을 엄격히 관리하는 고성능 수학적 컨테이너입니다.

핵심 속성 (Attributes)

- **Shape** (형상) 데이터의 차원과 크기를 정의합니다.
(예: 28x28 이미지 → (28, 28))
- **Dtype** (데이터 타입) 메모리 효율성을 위해 배열 내 모든 요소는 동일한 타입(**float32**, **int64** 등)을 가집니다.

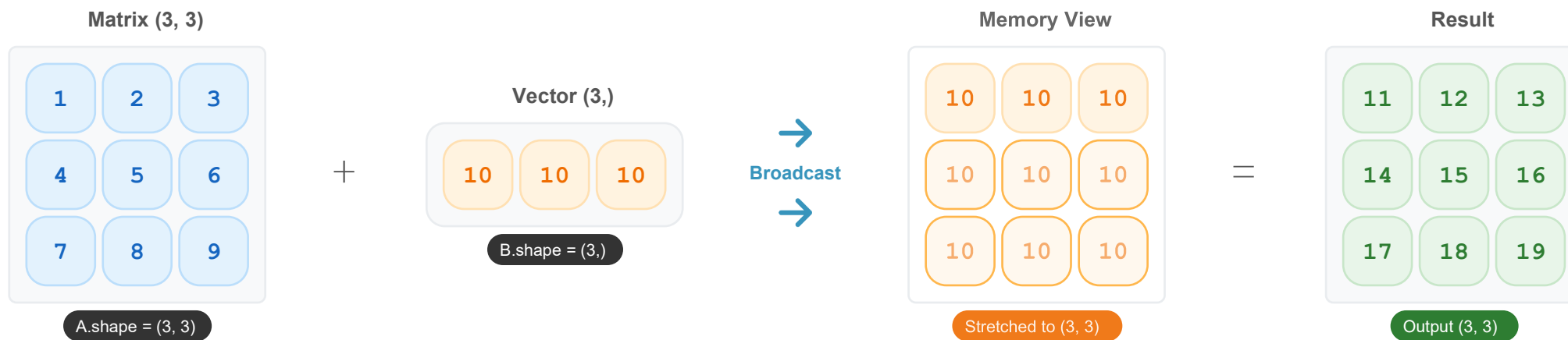
수학적 모델링 역할

- 벡터/행렬 연산 반복문 없이 수식 그대로 코드로 옮길 수 있는 벡터화 연산을 지원합니다.
- 다차원 데이터 표현 이미지(3D), 비디오(4D), 배치 데이터 등 복잡한 현실 데이터를 숫자로 구조화합니다.

"딥러닝의 관점에서, 텐서(**Tensor**)는 결국 '그래디언트(**Gradient**)'가 추가된 **ndarray**'입니다."

브로드캐스팅 (Broadcasting)

브로드캐스팅은 형상(Shape)이 다른 배열끼리 연산할 때, 작은 배열을 자동으로 확장하여 차원을 맞추는 NumPy의 핵심 기능입니다.

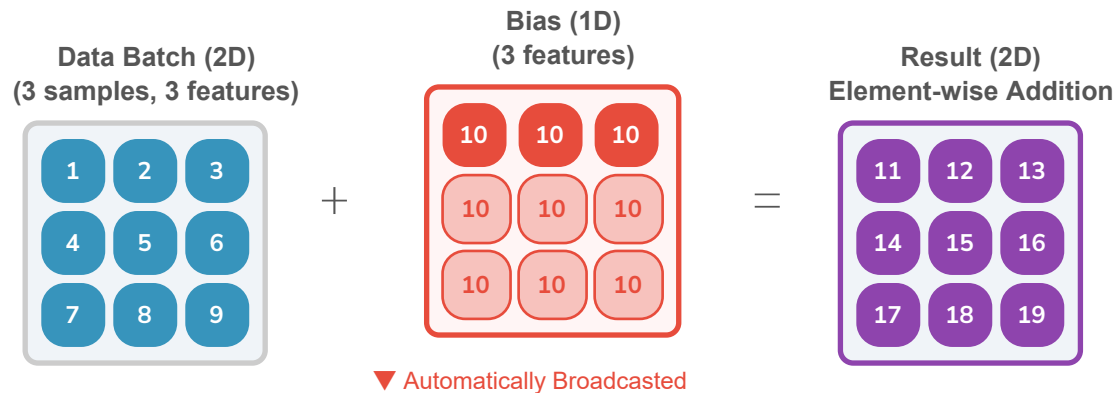


💡 왜 중요한가요?

데이터를 실제로 복사하지 않고(**No Copy**) 가상의 확장을 통해 연산을 수행하므로, 메모리를 절약하고 연산 속도가 매우 빠릅니다. 반복문(**for loop**)을 사용하지 않는 벡터화 연산의 핵심입니다.

1D + 2D 연산: 딥러닝의 Bias(편향) 더하기

```
batch_data + bias # (3, 3) + (3,) → (3, 3)
```



💡 **Insight:** 반복문 없이 수천 개의 데이터(Batch)에 편향(Bias)을 한 번에 더할 수 있어 학습 속도가 획기적으로 빨라집니다.

집계 함수의 역할

집계(Aggregation)는 다차원 배열을 단일 값(Scalar)이나 더 작은 차원으로 압축하는 과정으로, 딥러닝에서 '얼마나 틀렸는지(Loss)'를 판단하고 학습 방향을 결정하는 핵심 연산입니다.



np.sum()

전체 합계: 모든 요소의 값을 더합니다. 그래디언트 누적이나 전체 에너지 계산에 사용됩니다.



np.mean()

평균: 데이터의 중심 경향을 파악합니다. 손실 함수(MSE, CrossEntropy) 계산의 표준이 됩니다.



np.max()
np.min()

최대/최소: 가장 강한 특징을 추출(Pooling)하거나, 데이터의 범위를 확인하여 이상치를 탐지합니다.



np.std()
np.var()

표준편차/분산: 데이터가 얼마나 퍼져있는지 확인합니다.
데이터 정규화(Normalization) 및 가중치 초기화에 필수적입니다.

💡 핵심 연결: 딥러닝의 손실 함수(Loss Function)는 결국 예측값과 정답 사이의 차이를 하나의 숫자로 만드는 '집계 함수'입니다.

축(Axis)의 정확한 이해

다차원 배열 연산에서 '**축(Axis)**'은 데이터의 방향을 결정합니다.
2차원(행렬)을 기준으로 **0번** 축과 **1번** 축의 의미를 명확히 구분해야 합니다.

축의 방향 (Direction)

- **axis=0 (default)** 세로 방향 (↓) **행(Row)**을 따라 이동하며 연산합니다.
데이터 샘플 간의 집계(예: 전체 학생의 평균 점수)를 의미합니다.
- **axis=1** 가로 방향 (→) **열(Column)**을 따라 이동하며 연산합니다.
샘플 내 특징 간의 집계(예: 한 학생의 과목별 평균)를 의미합니다.

딥러닝 데이터 차원 매핑

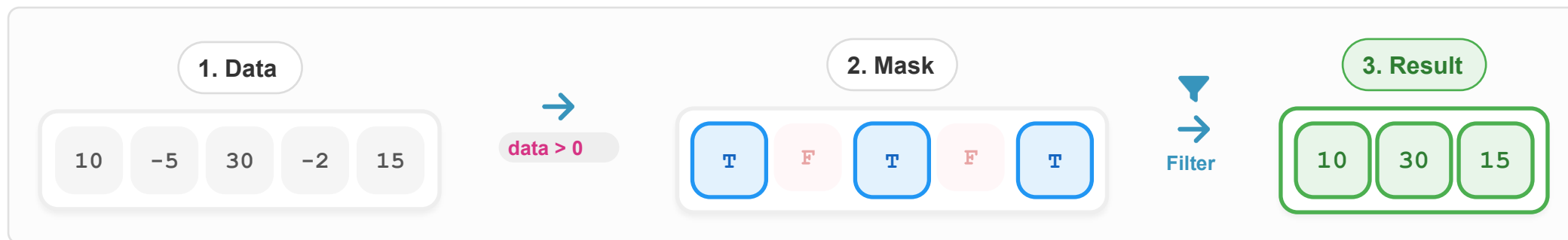
- 배치 차원 (**Batch Dimension**) 보통 **axis=0**에 해당합니다.
한 번에 처리하는 데이터 샘플의 개수(**N**)입니다.
- 특징 차원 (**Feature Dimension**) 보통 **axis=1** 이상에 해당합니다.
각 데이터가 가지는 속성의 개수(**D, C, H, W** 등)입니다.

"손실 함수(**Loss Function**)는 결국 전체 배치 데이터(**axis=0**)에 대해
하나의 스칼라 값을 내기 위한 집계 함수입니다."

13

불리언 인덱싱 (Boolean Indexing)

배열에 조건(Condition)을 적용하여 True/False로 구성된 마스크(Mask)를 만들고, 이를 통해 원하는 데이터만 필터링(Filtering)하는 강력한 기법입니다.



✖ 어디에 활용하나요?

- 노이즈 제거: 유효 범위를 벗어난 이상치(Outlier) 삭제

- 데이터 추출: 특정 클래스(예: `label==1`) 데이터만 선택

"데이터 전처리의 핵심 도구"

조건부 인덱싱을 활용하여 노이즈를 제거하고 결정 로직을 간결하게 구현할 수 있습니다.



이상치 제거

정상 범위를 벗어난
이상치(**Outlier**)를 찾아
제거하거나 **0**으로 보정합니다.



임계값 결정

예측 확률이 **0.5** 이상인
데이터만 **1(True)**로
변환하여 분류를 수행합니다.

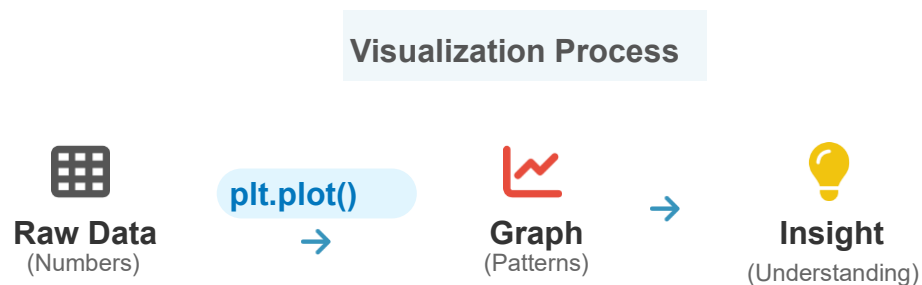


데이터 필터링

결측치(**NaN**)를 찾거나
특정 조건의 데이터만
추출하여 학습셋을 만듭니다.

Matplotlib: 데이터 시각화

모델의 훈련 과정과 데이터 구조를 직관적으로 파악하는 진단 도구입니다.



데이터 탐색 (EDA)

분포, 이상치, 경향성을 눈으로 확인

모델 진단 (Debugging)

손실 곡선, 과적합 여부, 학습 상태 모니터링

💡 숫자로만 보이는 데이터에 형태를 부여하여 직관적인 통찰을 얻습니다.

The screenshot shows the Matplotlib 3.10.8 documentation website. The header includes the Matplotlib logo, navigation links (Plot types, User guide, Tutorials, Examples, Reference, Contribute, Releases), a search icon, a version selector (Choose version), and social media icons. The main content area is titled "Matplotlib 3.10.8 documentation" and describes it as a comprehensive library for creating static, animated, and interactive visualizations. A sidebar on the left contains a "Cheatsheets" link with a thumbnail image and a "Support Matplotlib" button. The "Install" section features tabs for "pip", "conda", "pixi", "uv", and "other", with a code block showing the command `pip install matplotlib`. The "Learn" section has two columns: "How to use Matplotlib?" with links to "Quick start guide", "User guide", "Tutorials", and "Frequently Asked Questions"; and "What can Matplotlib do?" with links to "Plot types" and "Examples". A "Reference" section is partially visible at the bottom.

matplotlib

Plot types User guide Tutorials Examples Reference Contribute Releases

Choose version

Matplotlib 3.10.8 documentation

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations.

Install

pip conda pixi uv other

```
pip install matplotlib
```

Learn

How to use Matplotlib?

- [Quick start guide](#)
- [User guide](#)
- [Tutorials](#)
- [Frequently Asked Questions](#)

What can Matplotlib do?

- [Plot types](#)
- [Examples](#)

Reference

Top-level interfaces to create...

Cheatsheets

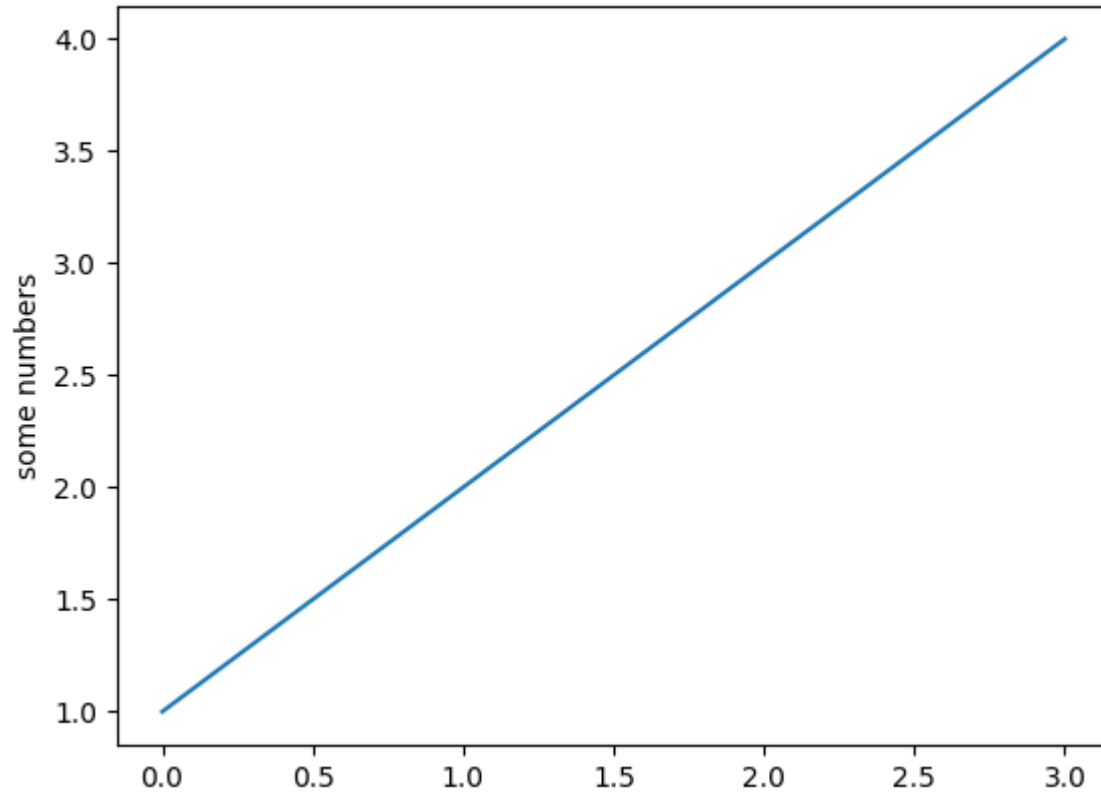
Support Matplotlib

On this page

- Install
- Learn
- Community
- What's new
- Contribute
- About us

<https://matplotlib.org/stable/>

Matplotlib: 선 그래프 (Line Plot)



```
import matplotlib.pyplot as plt
```

```
plt.plot([1, 2, 3, 4])
```

```
plt.ylabel('some numbers')
```

```
plt.show()
```

산점도 (Scatter Plot) 핵심 개념

산점도는 **x**축과 **y**축에 데이터를 점으로 찍어 두 변수 간의 관계를 시각적으로 보여줍니다.
머신러닝 데이터 분석에서 가장 먼저 확인해야 할 4가지 구조적 특징입니다.

상관관계 (Relationship)

변수 간의 비례/반비례 관계를 파악합니다.
(예: 학습 시간이 늘면 점수도 오르는가?)

군집 (Cluster)

데이터가 끼리끼리 뭉쳐 있는 그룹을 발견합니다.
(예: 고객 세분화, 뉴스 토픽 분류)

이상치 (Outlier)

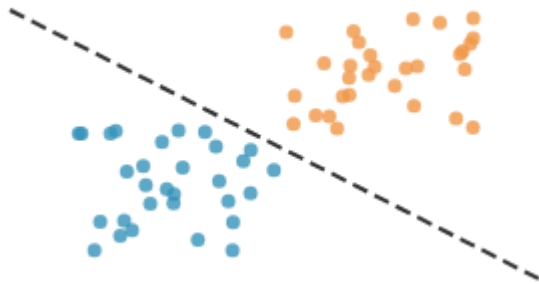
전체 패턴에서 벗어난 특이 데이터를 식별합니다.
(예: 센서 오류, 비정상 거래 탐지)

분리 가능성 (Separability)

클래스(정답)를 선이나 곡선으로 나눌 수 있는지 봅니다.
(예: 딥러닝 모델의 분류 난이도 판단)

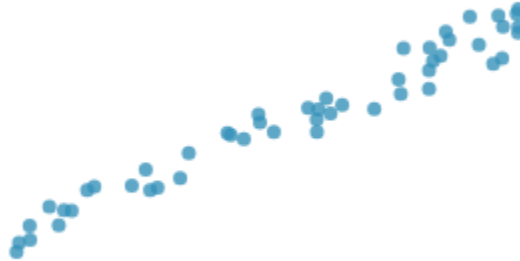
산점도는 머신러닝 모델의 결정 경계를 시각화하고 데이터의 특성을 파악하는 데 필수적

1. 결정 경계 (Decision Boundary)



데이터가 선형적으로 분리 가능한지
직관적으로 확인할 수 있습니다.

2. 특징 상관관계 (Correlation)



X와 Y 변수 간의
상관관계 강도를 파악합니다.

3. 이상치 탐지 (Anomaly Detection)



전체 패턴에서 벗어난
이상 데이터(Outlier)를 식별합니다.

i 분류 문제 해결을 위한 데이터 구조 파악의 첫 번째 단계입니다.

"그래프가 스스로를 설명하지 못하면 나쁜 그래프입니다." 제목, 축 라벨, 범례는 선택이 아닌 필수 요소입니다.

1. 필수 5대 요소 추가

```
plt.plot(x, y, label='Loss')
plt.title('Training Loss
Curve')
plt.xlabel('Epoch')
plt.ylabel('Loss Value')
plt.legend()
plt.grid(True)
plt.show()
```

2. 스타일 및 가독성 향상

```
# 스타일 테마 적용 (seaborn, ggplot 등)
plt.style.use('seaborn-v0_8')

# 그림 크기 및 해상도 조절
plt.figure(figsize=(10, 6), dpi=100)

# 여러 선 겹쳐 그리기
plt.plot(x, train_loss, 'b-', label='Train')
plt.plot(x, val_loss, 'r--', label='Val')

# 주 범위 지정 (필요 시)
plt.ylim(0, 1.0)
```

Tip: 딥러닝 학습 곡선에서는 Train/Validation 손실을 하나의 그래프에 겹쳐 그려 과적합(Overfitting) 여부를 판단합니다.



배열 사고 (Thinking in Arrays)

데이터 전체를 하나의 덩어리로 보고
반복문 없이 한 번에 처리하는 습관



벡터화 계산 (Vectorization)

브로드캐스팅을 통해 형상이 다른 배열도
자동으로 맞춰 고속으로 연산 수행



데이터 집계 (Aggregation)

축(Axis)을 이해하고 데이터를 요약하여
모델 학습의 신호(Loss)를 만드는 기초



시각화 해석 (Visualization)

단순한 그림이 아닌 데이터의 분포와
모델의 학습 상태를 진단하는 도구